Communication-Efficient Federated Natural Policy Gradient: A Reproduction and Extension of FedNPG-ADMM

Edoardo Scarpel
edoardo.scarpel@phd.unipd.it

Daniele Zebele
daniele.zebele@phd.unipd.it

Department of Information Engineering, University of Padua
October 12, 2026

Abstract

Federated Natural Policy Gradient (FedNPG) inherits the fast convergence of Natural Policy Gradient (NPG) but requires every agent to uplink its local Fisher information matrix $\mathbf{H}_i \in \mathbb{R}^{d\times d}$ each round. The resulting $\mathcal{O}(d^2)$ communication cost becomes prohibitive for deep policies. Lan et al. [8] reformulate the global natural-gradient direction as a distributed quadratic program solved by the Alternating Direction Method of Multipliers (ADMM), so that each agent transmits only two d-dimensional vectors, an $\mathcal{O}(d)$ cost, at the same convergence rate. We reproduce that method from scratch in PyTorch and evaluate it on Swimmer-v4, Hopper-v4 and Humanoid-v4 with $N \in \{1, 2, 4, 8\}$ agents over five random seeds. On Swimmer we match the reported final rewards at two, four and eight agents and exceed them with one; on Humanoid the method scales from 993 to 6329 average return as agents increase from one to eight. On Hopper, with a reduced configuration closer to the algorithm as written, four and eight agents reach 2536 and 2656, within one standard deviation of the reported values, and learn faster than one or two. Against the two baselines at eight agents, FedNPG-ADMM matches standard-average FedNPG on Hopper and exceeds it on Humanoid. We further quantify the communication saving directly, measuring reductions by factors of 2.4×10^3, 9.2×10^3 and 3.6×10^5 in transmitted parameters per agent per round on the three environments. Finally, we quantize the two vectors each agent transmits, which the original work does not attempt. Across five seeds, 8-bit stochastic quantization keeps both the final reward and the faster learning with more agents on Swimmer and Humanoid, for a further fourfold reduction in uplink against the `float32` values exchanged (eightfold at the paper's 64-bit accounting); at 6 bits Humanoid collapses at every agent count.

1 Introduction

Federated Reinforcement Learning (FedRL) lets several agents improve a shared policy without pooling their raw trajectories. In the *horizontal* setting studied here, every agent faces the same environment, collects its own experience, and periodically exchanges model information with a central server [10]. This differs from Multi-Agent Reinforcement Learning (MARL), where agents inhabit a single shared environment and interact with one another. No agent here observes another's actions.

First-order federated policy gradient methods communicate a d-dimensional gradient per round, which is cheap but inherits the slow convergence of plain policy gradient. Natural Policy Gradient (NPG) converges considerably faster by preconditioning the gradient with the inverse Fisher information matrix, but a naive federation of NPG requires each agent to transmit its full local Fisher $\mathbf{H}_i \in \mathbb{R}^{d\times d}$. For the policies used in continuous control this is fatal. Our Humanoid

policy has $d = 727\,074$ parameters, so a single $\mathbf{H}_i$ occupies 2.1 PB at double precision, four orders of magnitude beyond any machine we could deploy.

Lan et al. [8] resolve this by observing that the global NPG direction $(\sum_i \mathbf{H}_i)^{-1} \sum_i \mathbf{g}_i$ is the solution of a distributed quadratic program, which consensus ADMM solves while exchanging only d-dimensional vectors. The communication cost drops from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$ at the same convergence rate and with no additional sample complexity.

Contributions. This report makes four contributions.

1. An exposition of the theory: policy and natural gradients, the consensus reformulation and ADMM updates behind FedNPG-ADMM, and its convergence and communication guarantees (Sections 2–3).

2. A from-scratch PyTorch implementation of Algorithm 1 and a reproduction study across three MuJoCo environments, four agent counts and five seeds (Sections 4–5). The code is publicly available at `https://github.com/edos08/FedNPG-ADMM-FedRL`.

3. A comparison against the two baselines the original work uses: standard-average FedNPG at $\mathcal{O}(d^2)$ and first-order FedPPO at $\mathcal{O}(d)$ (Section 6).

4. Quantization of the agent-to-server transmissions, which is not part of the original paper and constitutes the novel contribution of this project (Section 7).

Two of our results depart from what the original leads a reader to expect. Our measured communication saving on Swimmer is about half an order of magnitude smaller than the figure the authors quote, and on Humanoid, standard-average FedNPG, which computes exactly the direction FedNPG-ADMM approximates, finishes well below it. Section 6 discusses both.

2 Background

2.1 Setting

N agents share a policy $\pi_{\boldsymbol{\theta}}$ with parameters $\boldsymbol{\theta} \in \mathbb{R}^d$ and each interacts with its own copy of the same Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$. The objective is the usual expected discounted return

$$J(\boldsymbol{\theta}) \;=\; \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}} \left[\textstyle\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \tag{1}$$

and the federation seeks $\boldsymbol{\theta}^\star = \arg\max_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ using every agent's samples while transmitting as little as possible.

2.2 Policy gradient

The policy gradient theorem [13, 14] expresses the gradient of J as an expectation over the policy's own trajectories,

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \;=\; \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}} [\textstyle\sum_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t)\, A^{\pi_{\boldsymbol{\theta}}}(s_t, a_t)], \tag{2}$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage of taking a in s. Each agent estimates (2) from its own batch $\mathcal{D}_i$ with the score-function estimator

$$\mathbf{g}_i \;=\; \frac{1}{|\mathcal{D}_i|} \sum_{\tau \in \mathcal{D}_i} \sum_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t)\, \hat{A}_t, \tag{3}$$

Table 1: Per-agent uplink per iteration, at the paper's convention of 64 bits per transmitted parameter. d is the policy parameter count for the architectures in Table 7. The dense Fisher is reported for scale; it is never materialized in our implementation (Section 6).

Environment	d	Dense $\mathbf{H}_i$	$\mathcal{O}(d^2)$ / $\mathcal{O}(d)$	Order
Swimmer-v4	4868	0.095 GB	2434	$10^{3.4}$
Hopper-v4	18 438	1.36 GB	9219	$10^{4.0}$
Humanoid-v4	727 074	2114 GB	363 537	$10^{5.6}$

which is line 7 of Algorithm 1. The advantages $\hat{A}_t$ come from Generalized Advantage Estimation (GAE) [12], an exponentially weighted sum of one-step temporal difference errors of a learned value function V,

$$\hat{A}_t \;=\; \sum_{l\geq 0} (\gamma\lambda)^l\, \delta_{t+l}, \qquad \delta_t \;=\; r_t + \gamma V(s_{t+1}) - V(s_t), \tag{4}$$

where $\lambda \in [0, 1]$ trades bias for variance. We use $\lambda = 0.95$, the value the paper states.

2.3 Natural policy gradient

Plain policy gradient ascends $\nabla_{\boldsymbol{\theta}} J$, which is sensitive to the parameterization. An equal step in parameter space can mean a negligible or a catastrophic change in the induced action distribution. NPG measures the step in distribution space instead. To second order, the average Kullback–Leibler (KL) divergence between the policy before and after a step Δ is a quadratic form in the Fisher information matrix,

$$\begin{aligned} \mathbb{E}_s\big[\mathrm{KL}\big(\pi_{\boldsymbol{\theta}}(\cdot \mid s) \,\big\|\, \pi_{\boldsymbol{\theta}+\Delta}(\cdot \mid s)\big)\big] \;&\approx\; \tfrac{1}{2}\,\Delta^\top \mathbf{F}(\boldsymbol{\theta})\,\Delta, \\ \mathbf{F}(\boldsymbol{\theta}) \;&=\; \mathbb{E}\Big[\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)\, \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)^\top\Big]. \end{aligned} \tag{5}$$

Maximizing the linearized objective $\mathbf{g}^\top\Delta$ subject to $\frac{1}{2}\Delta^\top\mathbf{F}\Delta \leq \delta$ gives the natural-gradient step

$$\boldsymbol{\theta} \;\leftarrow\; \boldsymbol{\theta} + \alpha\, \mathbf{F}^{-1}\mathbf{g}, \qquad \alpha \;=\; \sqrt{\frac{2\delta}{\mathbf{g}^\top \mathbf{F}^{-1}\mathbf{g}}}, \tag{6}$$

which is invariant to reparameterization and moves the policy by a KL divergence of δ per step [6, 11]. Single-agent NPG, and Trust Region Policy Optimization (TRPO) with it, is matrix-free: (6) needs only $\mathbf{F}^{-1}\mathbf{g}$, which conjugate gradient computes from products $\mathbf{F}v$, and each such product is a double backward pass through the KL divergence. $\mathbf{F}$ is never formed.

2.4 The federated bottleneck

Federating NPG naively means forming the global direction from every agent's local gradient $\mathbf{g}_i$ and local Fisher $\mathbf{H}_i$:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \sqrt{\frac{2N\delta}{\big(\sum_i \mathbf{g}_i\big)^\top \big(\sum_i \mathbf{H}_i\big)^{-1} \sum_i \mathbf{g}_i}}\, \Big(\sum_{i=1}^N \mathbf{H}_i\Big)^{-1} \sum_{i=1}^N \mathbf{g}_i. \tag{7}$$

This is what Lan et al. [8] call *standard-average FedNPG*. The matrix-free trick no longer applies. The server needs each agent's $\mathbf{H}_i$ to form the sum, and no agent can evaluate a Fisher-vector product on another's data. Each agent must therefore uplink $\mathcal{O}(d^2)$ numbers per round. Table 1 shows what that means concretely for the three environments studied here.

3 FedNPG-ADMM

3.1 The global direction as a consensus problem

The direction in (7) is the solution of a quadratic problem whose objective is a sum over agents [8]:

$$\mathbf{y}^\star \;=\; \Big(\sum_i \mathbf{H}_i\Big)^{-1}\sum_i \mathbf{g}_i \;=\; \arg\min_{\mathbf{y}} \sum_{i=1}^{N}\Big(\tfrac{1}{2}\mathbf{y}^\top\mathbf{H}_i\mathbf{y} - \mathbf{g}_i^\top\mathbf{y}\Big). \tag{8}$$

Each summand depends only on agent i's data, but all of them share the same $\mathbf{y}$. Giving every agent its own copy $\mathbf{y}_i$ and requiring the copies to agree turns (8) into the equivalent consensus problem

$$\min_{\mathbf{y},\{\mathbf{y}_i\}} \sum_{i=1}^{N}\Big(\tfrac{1}{2}\mathbf{y}_i^\top\mathbf{H}_i\mathbf{y}_i - \mathbf{g}_i^\top\mathbf{y}_i + \tfrac{\rho}{2}\|\mathbf{y}_i-\mathbf{y}\|^2\Big) \quad \text{s.t.} \quad \mathbf{y}_i = \mathbf{y},\ i=1,\dots,N, \tag{9}$$

where the penalty $\frac{\rho}{2}\|\mathbf{y}_i-\mathbf{y}\|^2$, with $\rho>0$, is zero on the feasible set and so leaves the solution unchanged. With dual variables $\boldsymbol{\lambda}_i \in \mathbb{R}^d$ for the constraints, the Lagrangian is

$$\mathcal{L}(\mathbf{y},\{\mathbf{y}_i\},\{\boldsymbol{\lambda}_i\}) \;=\; \sum_{i=1}^{N}\Big(\tfrac{1}{2}\mathbf{y}_i^\top\mathbf{H}_i\mathbf{y}_i - \mathbf{g}_i^\top\mathbf{y}_i + \tfrac{\rho}{2}\|\mathbf{y}_i-\mathbf{y}\|^2 + \boldsymbol{\lambda}_i^\top(\mathbf{y}_i-\mathbf{y})\Big). \tag{10}$$

3.2 The ADMM updates

The Alternating Direction Method of Multipliers [2] minimizes (10) over the local copies, then over the consensus variable, then takes a gradient-ascent step on the duals. Each update has a closed form.

Local step. Setting $\partial\mathcal{L}/\partial\mathbf{y}_i = \mathbf{H}_i\mathbf{y}_i - \mathbf{g}_i + \rho(\mathbf{y}_i-\mathbf{y}) + \boldsymbol{\lambda}_i = 0$ gives

$$\mathbf{y}_i \;=\; (\mathbf{H}_i+\rho I)^{-1}(\mathbf{g}_i - \boldsymbol{\lambda}_i + \rho\,\mathbf{y}), \tag{11}$$

which involves only agent i's own $\mathbf{H}_i$ and $\mathbf{g}_i$.

Consensus step. Setting $\partial\mathcal{L}/\partial\mathbf{y} = -\sum_i\big(\rho(\mathbf{y}_i-\mathbf{y})+\boldsymbol{\lambda}_i\big) = 0$ gives $\mathbf{y} = \frac{1}{N}\sum_i\mathbf{y}_i + \frac{1}{N\rho}\sum_i\boldsymbol{\lambda}_i$.

Dual step. $\boldsymbol{\lambda}_i \leftarrow \boldsymbol{\lambda}_i + \rho\,(\mathbf{y}_i-\mathbf{y})$.

The sum of the duals is conserved. Summing the dual step over agents adds $\rho\big(\sum_i\mathbf{y}_i - N\mathbf{y}\big)$, which is zero whenever $\mathbf{y}$ is the plain average of the $\mathbf{y}_i$. Starting from $\boldsymbol{\lambda}_i = \mathbf{0}$, the duals therefore always sum to zero, and the consensus step reduces to $\mathbf{y} = \frac{1}{N}\sum_i\mathbf{y}_i$: the server only ever averages. At a fixed point, where every $\mathbf{y}_i = \mathbf{y}$, the local step reads $\mathbf{H}_i\mathbf{y} = \mathbf{g}_i - \boldsymbol{\lambda}_i$, and summing over agents gives $\big(\sum_i\mathbf{H}_i\big)\mathbf{y} = \sum_i\mathbf{g}_i$, which is exactly the direction (8). The only quantities that cross the network are $\mathbf{y}_i$ and $\mathbf{g}_i$, two d-dimensional vectors per agent.

FedNPG-ADMM does not run ADMM to convergence within an iteration. It performs a single round per policy iteration and carries $\mathbf{y}$ and $\boldsymbol{\lambda}_i$ over to the next one, so the ADMM iterates track a target that moves slowly as $\boldsymbol{\theta}$ changes. Algorithm 1 lists the result; Theorem 1 below shows that one round per iteration is enough.

Algorithm 1 FedNPG-ADMM (Algorithm 1 of 8)

1: **Initialize** $\boldsymbol{\theta}^0$, $\mathbf{y}^0 = \mathbf{0}$, $\mathbf{y}_i^0 = \mathbf{y}^0$, $\boldsymbol{\lambda}_i = \mathbf{0}$ for all i
2: **for** $k = 0, 1, \ldots, K-1$ **do**
3: Server broadcasts $\boldsymbol{\theta}^k$ and $\mathbf{y}^{k-1}$ to all agents
4: **for** each agent i **in parallel do**
5: Collect a rollout of T timesteps under $\pi_{\boldsymbol{\theta}^k}$
6: $\boldsymbol{\lambda}_i \leftarrow \boldsymbol{\lambda}_i + \rho\,(\mathbf{y}_i - \mathbf{y}^{k-1})$ ▷ dual update
7: $\mathbf{g}_i \leftarrow \frac{1}{|\mathcal{D}_i|}\sum_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t)\,\hat{A}_t$ ▷ score-function gradient
8: $\mathbf{y}_i \leftarrow (\mathbf{H}_i + \rho I)^{-1}(\mathbf{g}_i - \boldsymbol{\lambda}_i + \rho\,\mathbf{y}^{k-1})$ ▷ local solve, by conjugate gradient
9: Send $\mathbf{y}_i, \mathbf{g}_i$ to the server ▷ $\mathcal{O}(d)$
10: **end for**
11: $\mathbf{y}^k \leftarrow \frac{1}{N}\sum_i \mathbf{y}_i$
12: $\boldsymbol{\theta}^{k+1} \leftarrow \boldsymbol{\theta}^k + \eta\sqrt{\dfrac{2N\delta}{(\sum_i \mathbf{g}_i)^\top \mathbf{y}^k}}\;\mathbf{y}^k$
13: **end for**

3.3 Algorithm

Line 8 is the local step (11), and it is where the $\mathcal{O}(d)$ claim is earned. The system $(\mathbf{H}_i + \rho I)\mathbf{y}_i = \mathbf{g}_i - \boldsymbol{\lambda}_i + \rho\mathbf{y}^{k-1}$ is solved by Conjugate Gradient (CG), which needs only Fisher-vector products on agent i's own data, so $\mathbf{H}_i$ is never formed. The ρI term is simultaneously the ADMM penalty and a Levenberg–Marquardt damping that keeps the system well-conditioned.

Line 12's step size carries a factor N that the single-agent step (6) lacks, and the factor is what makes the trust region refer to the average agent. At exact consensus, $\mathbf{y}^k$ solves (8), hence $(\sum_i \mathbf{g}_i)^\top\mathbf{y} = \mathbf{y}^\top(\sum_i \mathbf{H}_i)\mathbf{y}$. Constraining the *average* per-agent KL to δ means $\frac{1}{2}\alpha^2\mathbf{y}^\top(\frac{1}{N}\sum_i \mathbf{H}_i)\mathbf{y} \le \delta$, which rearranges exactly to the $\sqrt{2N\delta/(\sum_i \mathbf{g}_i)^\top\mathbf{y}}$ of line 12. The denominator $(\sum_i \mathbf{g}_i)^\top\mathbf{y}$ grows linearly with the number of agents, and the N in the numerator cancels that growth, so every iteration moves the policy by the same KL divergence δ whatever N is. More agents therefore give a better-estimated direction, not a larger step.

3.4 Convergence guarantee

Lan et al. [8] show that the single ADMM round loses nothing in convergence rate. They assume a bounded score function, $\|\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)\| \le G$, that is M-Lipschitz in $\boldsymbol{\theta}$, bounded rewards $r(s,a) \in [0, R]$, and a Fisher matrix bounded away from singularity, $\mathbf{F}(\boldsymbol{\theta}) \succeq \mu_F I$ for some $\mu_F > 0$. Writing L_J for the smoothness constant of J, their result is the following.

Theorem 1 (Theorem 4.3 of 8)**.** *For a target error ϵ, let each agent sample $\mathcal{O}(1/((1-\gamma)^4 N\epsilon))$ trajectories per iteration, and set $\eta = \mu_F^2/(4G^2(56G^2 + L_J))$. After $K = \mathcal{O}(1/((1-\gamma)^2\epsilon))$ iterations,*

$$\frac{1}{K}\sum_{k=1}^{K} \mathbb{E}\big\|\nabla J(\boldsymbol{\theta}^k)\big\|^2 \;\le\; \epsilon. \tag{12}$$

The proof shows that the gap between the ADMM direction $\mathbf{y}^k$ and the exact natural-gradient direction shrinks geometrically from one iteration to the next, down to a floor set by the statistical error of the sampled gradients, which decays with the batch size. One ADMM round per iteration therefore inherits the iteration complexity of exact NPG. Table 2 puts this together per agent. The factor N in the per-agent sample complexity is the benefit of federation: each agent collects $1/N$ of the data a single learner would need. The communication complexity is K rounds of $2d$ values, against K rounds of d^2 for standard-average FedNPG.

Table 2: Per-agent complexity to reach an ϵ-stationary point, from Table 1 of Lan et al. [8].

	NPG	Standard-average FedNPG	FedNPG-ADMM
Samples	$\mathcal{O}\Big(\frac{1}{(1-\gamma)^6\epsilon^2}\Big)$	$\mathcal{O}\Big(\frac{1}{(1-\gamma)^6 N\epsilon^2}\Big)$	$\mathcal{O}\Big(\frac{1}{(1-\gamma)^6 N\epsilon^2}\Big)$
Communication	—	$\mathcal{O}\Big(\frac{d^2}{(1-\gamma)^2\epsilon}\Big)$	$\mathcal{O}\Big(\frac{d}{(1-\gamma)^2\epsilon}\Big)$

The theorem's η depends on constants (G, L_J, μ_F) that are not known for a neural policy, so in practice η is a hyperparameter (Section 4).

3.5 A property of the single-agent case

One consequence of Algorithm 1 matters for interpreting $N = 1$ results and is not remarked on in the original. With one agent, the broadcast consensus variable is that agent's own previous solve, $\mathbf{y}^{k-1} = \mathbf{y}_1^{k-1}$, so line 6 reads $\boldsymbol{\lambda}_1 \leftarrow \boldsymbol{\lambda}_1 + \rho(\mathbf{y}_1 - \mathbf{y}_1) = \boldsymbol{\lambda}_1$ and the dual variable never leaves its zero initialization. Algorithm 1 at $N = 1$ therefore reduces exactly to

$$\mathbf{y} \;=\; (\mathbf{H} + \rho I)^{-1}(\mathbf{g} + \rho\,\mathbf{y}^{k-1}), \tag{13}$$

damped NPG with a momentum term, exercising none of the consensus machinery. We verified this numerically: $\|\boldsymbol{\lambda}_1\| = 0$ to machine precision at every iteration for $N = 1$, and non-zero from the second iteration onwards for $N > 1$. Any $N = 1$ number in this report, and in the original, should be read with that in mind.

4 Implementation and experimental setup

We implemented FedNPG-ADMM from scratch in PyTorch, as no reference code is publicly available; ours is, at `https://github.com/edos08/FedNPG-ADMM-FedRL`. Federated training is simulated in one process with N agents, each with its own copy of the environment. Every agent-to-server transmission passes through a single channel that counts bytes at the paper's convention of 64 bits per value, although the tensors themselves are `float32`, so the three algorithms of Section 6 are metered by the same code, and the quantizers of Section 7 plug in at the same point.

Hyperparameters follow Table 4 of the paper (Appendix A). Two values are our own reading. The step multiplier η depends on constants that are unknown for a neural policy (Theorem 1), so we set $\eta = 1$ and take the ADMM step unscaled. Table 4's "Learning rate" column we use as the critic's Adam learning rate, since the actor's step is fixed by the trust region.

The paper trains with "the practical settings in TRPO with line search and in stable-baselines3", which Algorithm 1 does not spell out. Swimmer and Humanoid use the standard versions: running observation and reward normalization, advantage normalization, orthogonal initialization, a backtracking line search and a clamp on $\log\sigma$ (Table 8). Hopper needed adjustments specific to the environment: it runs without reward, advantage and observation normalization, line search and the $\log\sigma$ clamp, which keeps it closer to Algorithm 1 as written. On each environment the baselines use the same settings as FedNPG-ADMM.

Every configuration runs with five seeds. Following the paper, we report the average return of the episodes completed in each iteration's batch, plotted against iterations, and summarize a run by its mean over the last 100 iterations.

Table 3: Final rewards, mean ± standard deviation across five seeds. Original values are from Table 2 of Lan et al. [8], computed over ten seeds. Humanoid does not appear in their Table 2. The Hopper row uses the configuration of Section 4.

Environment	Source	$N = 1$	$N = 2$	$N = 4$	$N = 8$
Swimmer-v4	Ours	$\mathbf{123.9 \pm 2.3}$	126.0 ± 1.3	127.0 ± 0.8	$\mathbf{128.4 \pm 0.8}$
	Paper	109.4 ± 5.9	123.6 ± 10.3	127.2 ± 2.2	128.5 ± 1.9
Hopper-v4	Ours	1941 ± 494	1636 ± 282	2536 ± 116	$\mathbf{2656 \pm 286}$
	Paper	1473 ± 383	2384 ± 371	2507 ± 230	2719 ± 173
Humanoid-v4	Ours	993 ± 187	3391 ± 796	5343 ± 121	$\mathbf{6329 \pm 334}$

5 Reproduction results

We run $N \in \{1, 2, 4, 8\}$ on Swimmer-v4, Hopper-v4 and Humanoid-v4 with five random seeds per configuration (60 runs). Swimmer and Humanoid use the configuration listed in Appendix A; Hopper uses a reduced one (Section 4). Table 3 reports the mean of each seed's final 100 iterations, then the mean and standard deviation across seeds, the same quantity Table 2 of the original reports.

5.1 Swimmer: reproduced

Figure 1(a) shows the curves behind the Swimmer column, and Figure 2(a) the same runs against uplink traffic. Final rewards fall within the reported spread at $N = 2, 4, 8$ and above it at $N = 1$ (123.9 against 109.4 ± 5.9), and they increase monotonically with N, as the paper describes. More agents also learn faster: at iteration 50 the smoothed return is 44, 62, 75 and 105 for $N = 1, 2, 4, 8$.

5.2 Humanoid: strong scaling

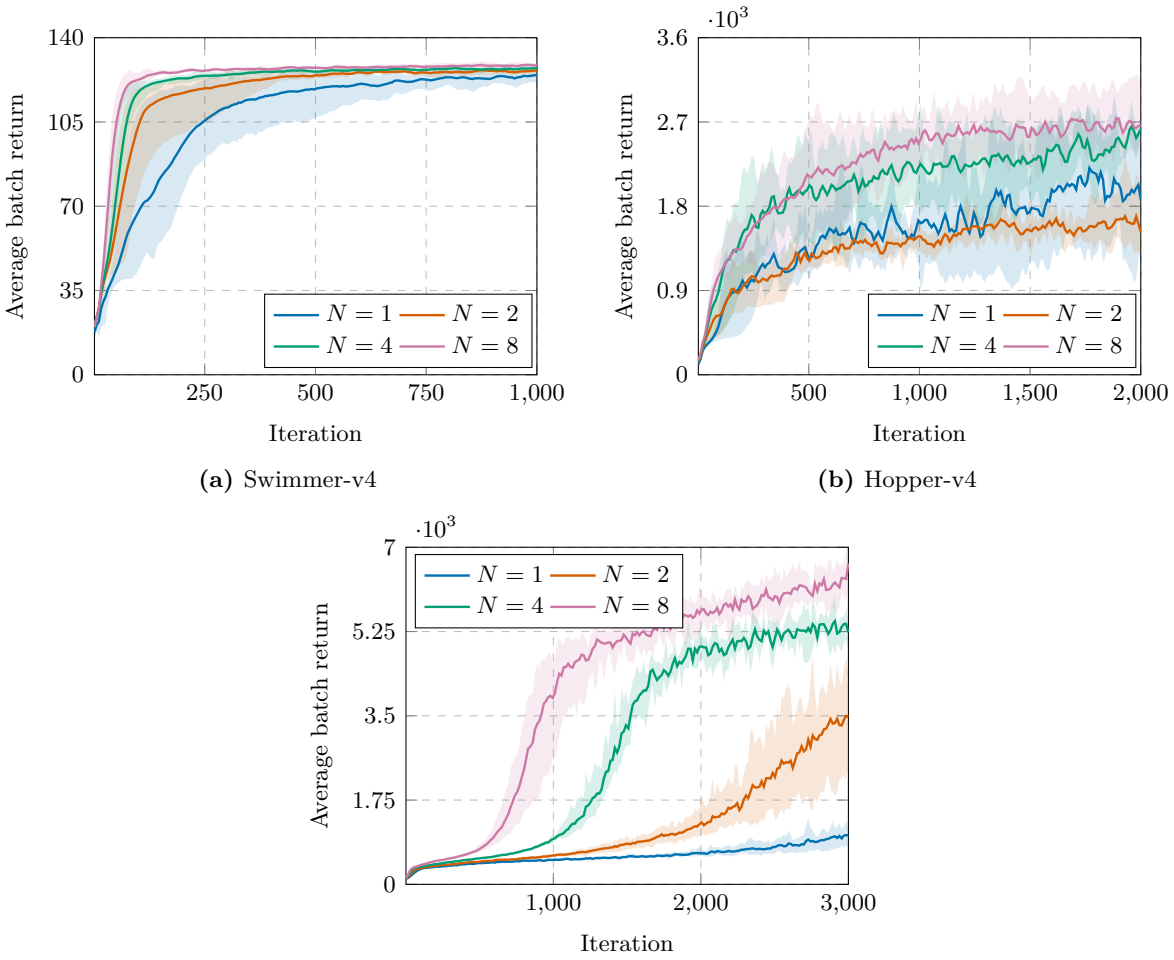
Humanoid is absent from the original Table 2, so we have no direct comparison. The qualitative claim nevertheless holds, since return grows 6.4× from $N = 1$ to $N = 8$, monotonically, with the $N = 4$ and $N = 8$ variance small relative to the mean.

5.3 Hopper

Four and eight agents reach 2536 ± 116 and 2656 ± 286 (Table 3), within one standard deviation of the reported 2507 and 2719, and learning speed increases with N as the paper describes. At iteration 100 the smoothed training return in Figure 1(b) is 598, 638, 986 and 1092 for $N = 1, 2, 4, 8$. The ordering is not strictly monotone throughout: two agents finish below one (1636 ± 282 against 1941 ± 494), and four agents briefly lead eight around iteration 200. Both gaps are smaller than the seed spread. What the five seeds resolve is two groups, one and two agents against four and eight, separated by roughly 700 at the end of training. Our single-agent value is above the reported 1473.

6 Baseline comparison

The original work compares three methods at $N = 8$: standard-average FedNPG (7), FedNPG-ADMM, and first-order federated Proximal Policy Optimization (FedPPO) with clipping parameter 0.2. We implement all three in the same codebase, holding everything except the server-side aggregation identical: same environments, normalization, GAE, critic and logged metrics. On Hopper all three methods use the reduced configuration of Section 4.



(a) Swimmer-v4

(b) Hopper-v4

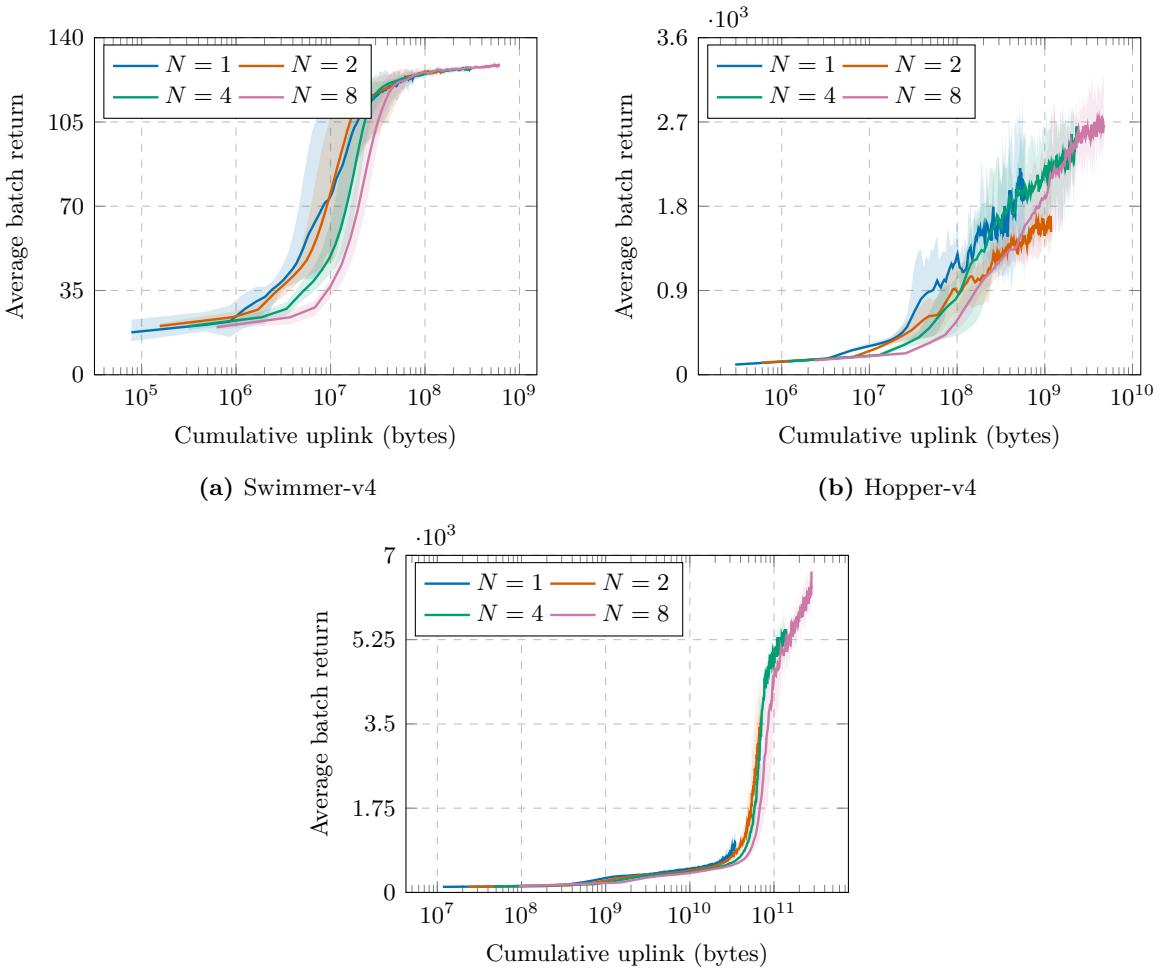
(c) Humanoid-v4

Figure 1: Convergence for each agent count. The solid line is the mean over five seeds and the shaded band spans the minimum and maximum seed, so the band shows the full observed range and not a modelled interval. Curves are smoothed with a centred 21-iteration rolling mean before aggregation, following Achiam [1], who smooth the same quantity over eleven epochs. Swimmer and Humanoid separate cleanly by agent count. On Hopper, four and eight agents rise faster and finish higher than one and two, while within each pair the bands overlap (Section 5.3).

6.1 Standard-average FedNPG

This arm solves (7) exactly by conjugate gradient on the pooled Fisher, exploiting $\sum_i \mathbf{H}_i v = N \cdot \mathbf{F}_{\text{pooled}} v$. The solve is matrix-free, but the *metering* is not: the channel is charged the full d^2 elements per agent per round.

The obvious objection is that a baseline meant to pay an $\mathcal{O}(d^2)$ cost ought to pay it. Materializing $\sum_i \mathbf{H}_i$ is impossible at Humanoid scale (Table 1: 2.1 PB), so a literal implementation would not exist for the environment where the $\mathcal{O}(d^2)$ cost matters most. The claim under test concerns *uplink bytes*, not server FLOPs; the paper measures "the number of transmitted parameters with double precision in each agent". We therefore compute the identical direction cheaply and report the true transmission cost. Without that metering the baseline would look as cheap as FedNPG-ADMM, and the comparison would measure nothing.



(a) Swimmer-v4

(b) Hopper-v4

(c) Humanoid-v4

Figure 2: The same runs plotted against cumulative uplink traffic instead of iteration count, on a logarithmic axis. The horizontal axis is the total across the federation: each agent transmits $2d$ values per iteration whatever N is, so a per-agent axis would place all four series on top of one another; that per-agent invariance is the $\mathcal{O}(d)$ property. On the federation-wide axis the curves shift right as N grows, by the traffic the extra agents add.

6.2 FedPPO

The paper names FedPPO and gives its clipping parameter, but specifies neither the aggregation scheme nor the local epoch count. We implement the FedAvg reading: each agent runs local PPO epochs from the broadcast $\boldsymbol{\theta}$ and uplinks its parameter delta, and the server averages. The alternative, a single clipped-surrogate gradient step per round, would parallel Algorithm 1's structure more neatly, but it strips PPO of the multi-epoch minibatch reuse that its clipping exists to make safe, so a comparison against it would not reflect PPO as it is normally run.

6.3 Communication cost, measured

These measurements let us check the paper's headline saving. Per agent per round, FedNPG-ADMM transmits 2434 times less than standard-average FedNPG on Swimmer and 363 537 times less on Humanoid (Table 1). Lan et al. [8] report about six orders of magnitude on Humanoid, which matches, and four on Swimmer, where we measure $10^{3.4}$. Part of the gap is that FedNPG-ADMM sends two vectors per round, so the saving is about $d/2$ rather than d; the paper does not say how its own figure was computed.

Table 4: Measured per-iteration uplink, Swimmer-v4 with four agents ($d = 4868$), at 64 bits per parameter. Measured values match the closed form exactly.

Method	Transmitted	Bytes/iteration	Relative
Standard-average FedNPG	$\mathbf{H}_i$ (d^2) + $\mathbf{g}_i$	758 473 344	2434×
FedNPG-ADMM	$\mathbf{y}_i$, $\mathbf{g}_i$ (2 vectors)	311 552	2×
FedPPO	$\Delta\boldsymbol{\theta}_i$ (1 vector)	155 776	1×

Table 5: Final rewards at $N = 8$, mean ± standard deviation across five seeds, computed as in Table 3. Hopper uses the configuration of Section 4 for all three methods.

Method	Swimmer-v4	Hopper-v4	Humanoid-v4
FedNPG-ADMM	128.4 ± 0.8	$\mathbf{2656 \pm 286}$	$\mathbf{6329 \pm 334}$
Standard-average FedNPG	127.1 ± 0.5	2494 ± 677	4699 ± 298
FedPPO	$\mathbf{129.8 \pm 0.7}$	1571 ± 588	5780 ± 158

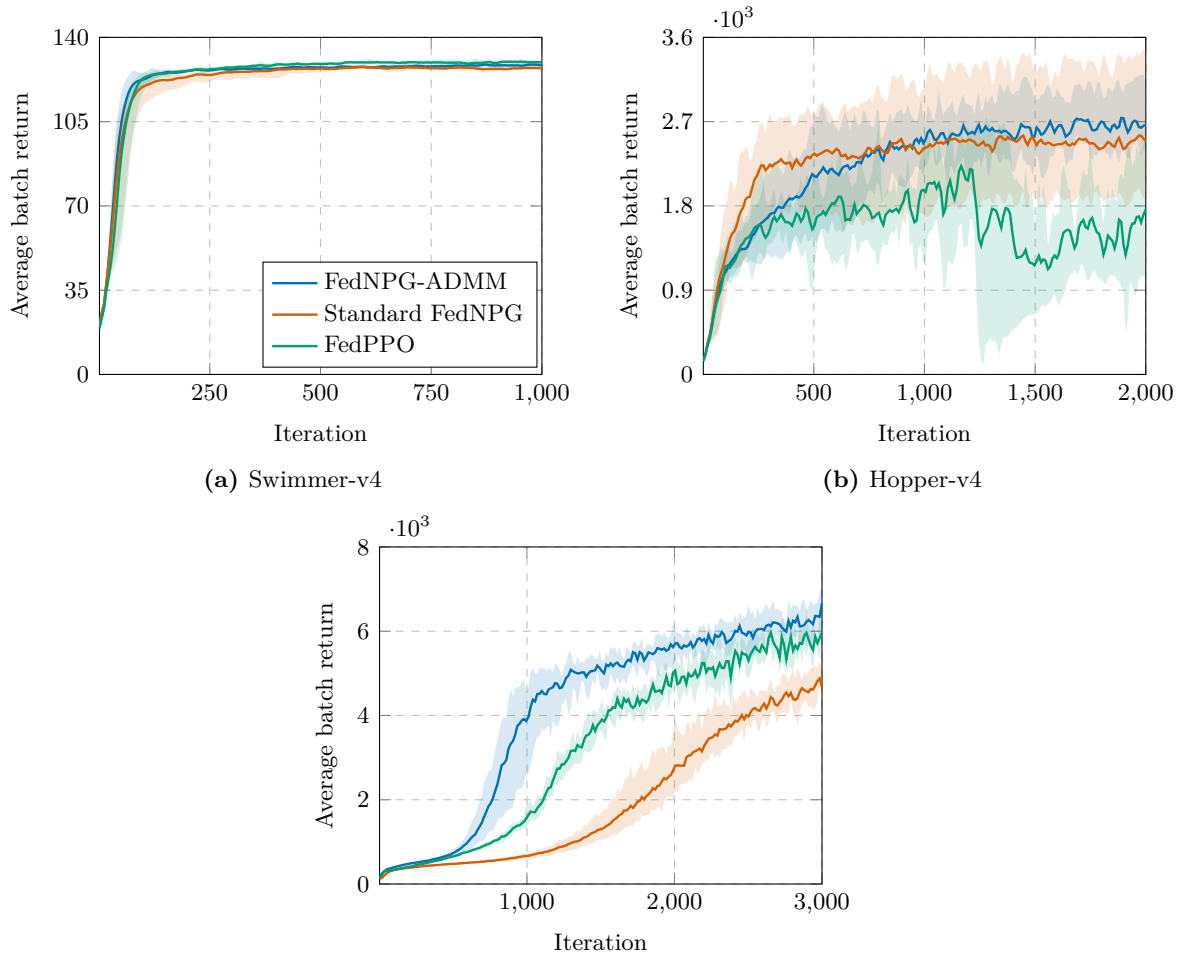
Table 4 also shows that FedNPG-ADMM is not the cheapest of the three methods, although the paper's framing invites a reader to assume it is. It is $\mathcal{O}(d)$, but with twice the constant of FedPPO, since it transmits two vectors per round where FedPPO transmits one. The argument for the method therefore rests on what it achieves per round rather than on what it costs per round, and a figure plotted against iterations cannot show that. We consequently plot reward against *cumulative bytes*, which is the axis on which the two effects can be read together.

6.4 Results

Table 5 and Figure 3 compare the methods per iteration. On Swimmer the three are indistinguishable: they finish within three points of each other, and the environment saturates near 128 for every method, as it does for every agent count in Section 5. On Hopper, FedNPG-ADMM and standard-average FedNPG agree to within the seed spread (2656 ± 286 against 2494 ± 677), which is the paper's central empirical claim: the ADMM approximation costs no reward relative to the exact aggregation it replaces, here at 9219 times less uplink per round. FedPPO finishes well below both, at 1571 ± 588.

Humanoid departs from that pattern. Standard-average FedNPG, which computes exactly the pooled natural-gradient direction that FedNPG-ADMM approximates with a single ADMM round, finishes *below* it, 4699 ± 298 against 6329 ± 334, a gap several times the seed spread. We have not established why. One candidate is that the single-round iterate is not a pure approximation: through the dual variables and the $\rho\,\mathbf{y}^{k-1}$ term of line 8, $\mathbf{y}^k$ carries information from previous iterations, which acts somewhat like momentum.

Figure 4 plots the same curves against uplink traffic. Against standard FedNPG the comparison is not close on any environment. Against FedPPO, which transmits half as much per round, the outcome depends on the environment. At the total traffic FedPPO uses over its whole run, FedNPG-ADMM has completed half as many iterations: on Swimmer both stand at about 128; on Hopper FedNPG-ADMM is ahead (2471 against 1763 on the smoothed curves); on Humanoid FedPPO is ahead (5973 against 5130). Per byte, then, FedNPG-ADMM is not uniformly the better method. Its advantage over FedPPO is in reward per round, which it holds on Hopper and Humanoid, and the reduction it delivers is the one the paper claims, against the $\mathcal{O}(d^2)$ method it was designed to replace.

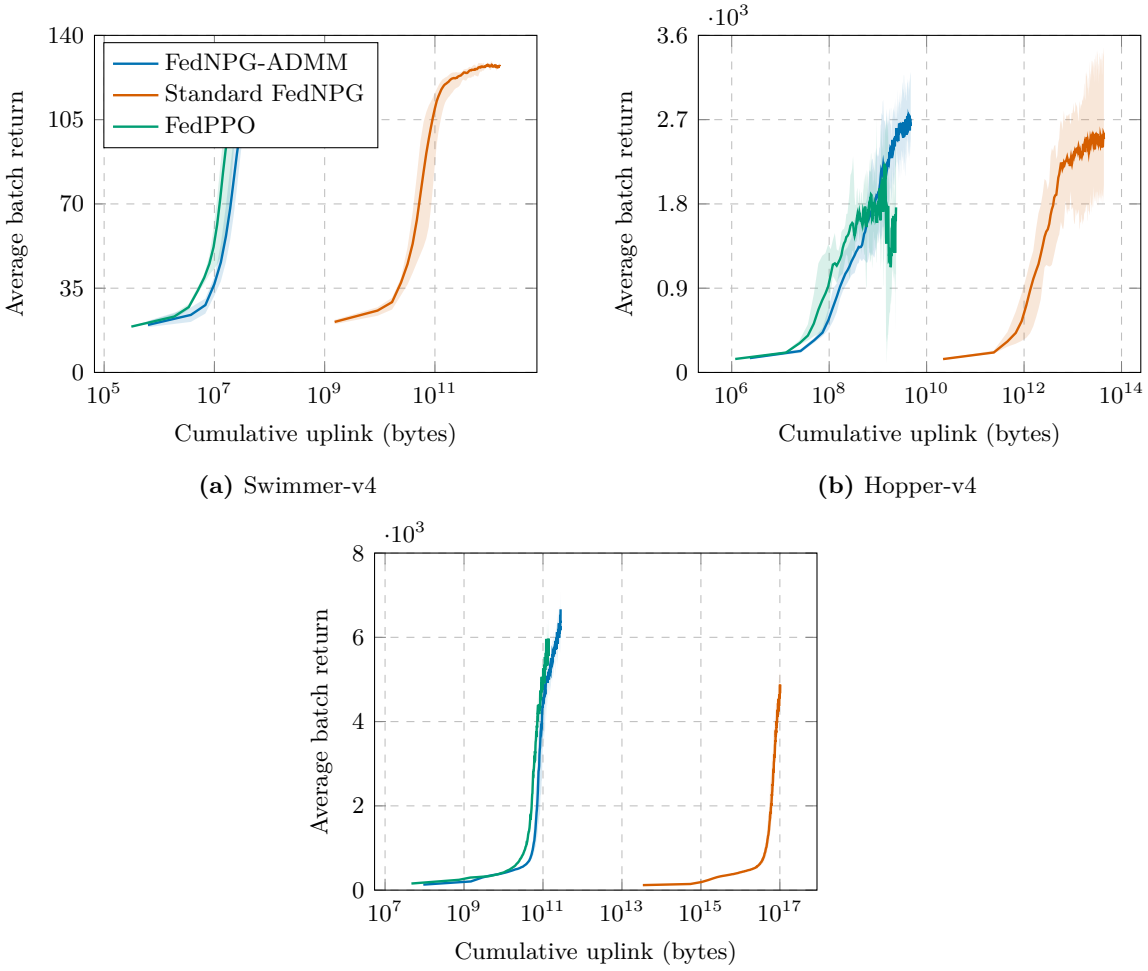


(a) Swimmer-v4

(b) Hopper-v4

(c) Humanoid-v4

Figure 3: The three methods at $N = 8$ against iteration. Mean over five seeds, band from minimum to maximum seed, smoothed as in Figure 1. Colours as in the legend of panel (a).



(a) Swimmer-v4

(b) Hopper-v4

(c) Humanoid-v4

Figure 4: The same runs against cumulative uplink traffic across the federation, on a logarithmic axis, at 64 bits per transmitted value. Standard-average FedNPG sits three to six orders of magnitude to the right because each agent uplinks its $d \times d$ Fisher matrix every round. Colours as in the legend of panel (a).

7 Quantizing the agent-to-server transmissions

FedNPG-ADMM reduces communication by shrinking *what* is transmitted from a matrix to two vectors. A second axis, which the original work does not explore, is to shrink *how precisely* those vectors are transmitted. The paper reports overhead at 64 bits per parameter, and our byte counter follows it, although the vectors our implementation exchanges are `float32`. Compression ratios below are therefore given against both references: the paper's 64-bit accounting and the 32 bits actually sent. If performance survives at 8 or 4 bits, the saving compounds with the $\mathcal{O}(d^2) \to \mathcal{O}(d)$ one.

This is the novel contribution of the project. The precedent is Q-GADMM [3], which Lan et al. [8] cite themselves, and which quantizes the vectors an ADMM scheme transmits in the *supervised* setting. We carry that idea into the reinforcement learning setting, where the transmitted vectors are gradient and consensus directions computed from a non-stationary sampling distribution. We found no prior work quantizing agent-to-server transmissions for federated policy gradient with continuous-control experiments. The nearest results are for compressed temporal difference learning with linear function approximation [9], which is theory-only and not policy gradient.

7.1 Design

A quantizer is a `Transform`: `tensor` $\to$ (`tensor`, `bits_per_element`), attached to the `Channel` through a factory that `train()` accepts. It quantizes and dequantizes in one call and declares the bit width the wire would have needed. No file in the algorithm proper knows quantization exists.

Two properties make the experiment measure something. The degraded tensor is what the server aggregates, because downstream code uses the return value of `send` and never the original, so the quantization error enters the update to $\boldsymbol{\theta}$. And the declared `bits_per_element` is what the byte meter uses, so the cost axis moves with the fidelity. Per iteration, with R ADMM rounds, N agents and b bits, the uplink is

$$\text{bytes} \;=\; N\,(1+R)\,d\,b/8, \tag{14}$$

since $\mathbf{g}_i$ is sent once per iteration and $\mathbf{y}_i$ once per round. Measured byte counts match (14) exactly; Figure 6 shows what they add up to over a full training run.

Stochastic uniform quantization. For a tensor x at b bits, with $L = 2^b - 1$ levels and step $s = (\max x - \min x)/L$, the value is placed at $\lfloor u \rfloor + \text{Bernoulli}(u - \lfloor u \rfloor)$ where $u = (x - \min x)/s$. Rounding stochastically instead of to nearest makes the estimator unbiased, $\mathbb{E}[\hat{x}] = x$, with per-element variance at most $s^2/4$. That property is the reason to prefer it here: a deterministic rounding error is systematic and would accumulate across iterations, whereas an unbiased one leaves the server receiving the correct gradient and consensus direction in expectation. The quantizer draws from a dedicated `torch.Generator` rather than the global stream, so runs at different bit widths see identical action-sampling sequences and differ only in the quantization.

7.2 Results

The main experiment is a five-seed grid: stochastic quantization at 8 and 6 bits, for $N \in \{2, 4, 8\}$, on all three environments (90 runs). One agent has no uplink to quantize, so $N = 1$ is unquantized throughout. The runs use the original configuration on every environment, Hopper included, and were made with the same settings and seeds as the unquantized runs they are compared against: the single-agent Swimmer runs of the two code bases agree to the last logged digit. Table 6 and Figure 5 give the results; a finer single-seed scan over bit widths at $N = 8$ locates the threshold more precisely.

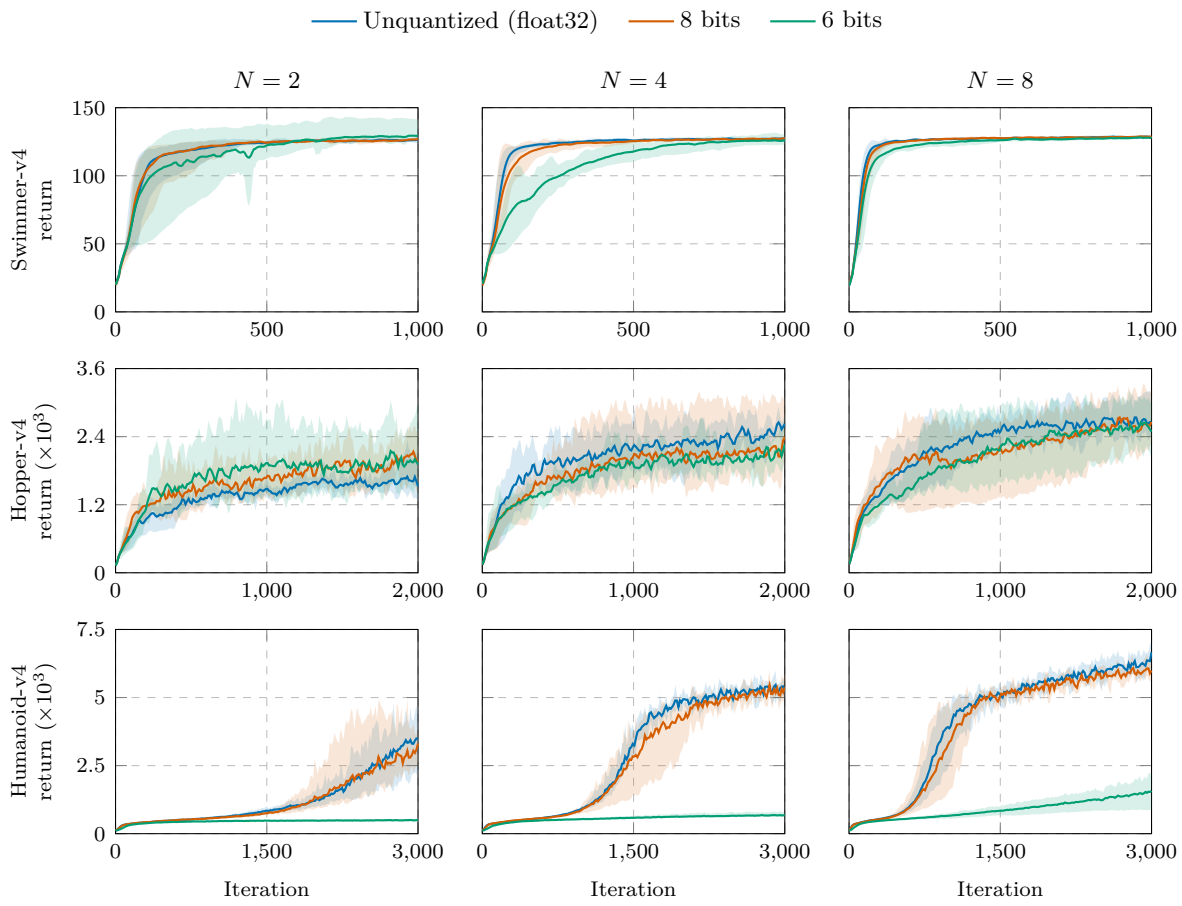


Figure 5: Return over training, unquantized (`float32`) and under stochastic quantization at 8 and 6 bits. Rows: environments; columns: number of agents N. Lines are the mean over five seeds, smoothed with a centred 21-iteration rolling mean; bands span the minimum to the maximum across seeds. Hopper uses its own configuration (Section 4) in all three series, Swimmer and Humanoid the original one.

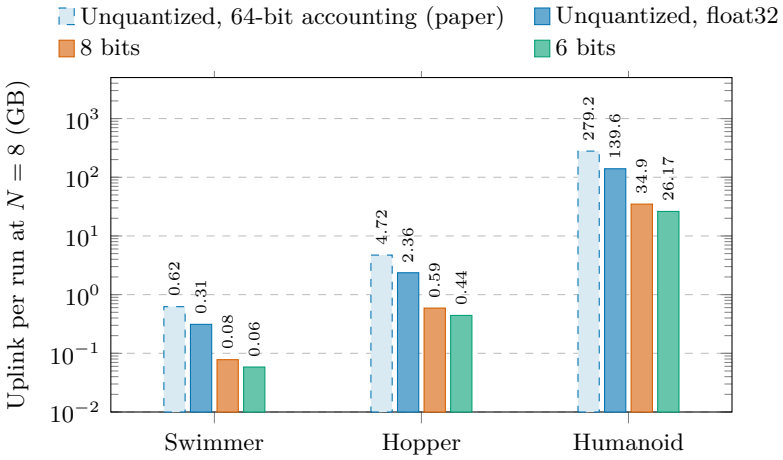


Figure 6: Uplink of one complete training run at $N = 8$, summed over agents; log scale. The unquantized run is shown both as metered, at the paper's 64 bits per value, and at the `float32` it actually exchanges, half of that. Measured byte counts, identical across seeds. Eight and six bits cut the uplink by 4× and 5.3× against `float32`, or 8× and 10.7× against the 64-bit accounting, on every environment; the totals scale linearly with N.

Table 6: Final reward under stochastic quantization, mean ± standard deviation across five seeds, computed as in Table 3. The unquantized column exchanges `float32` values; 8 and 6 bits cut the uplink per iteration by 4× and 5.3× against it, or 8× and 10.7× at the paper's 64-bit accounting. Hopper uses the original configuration, so its unquantized column is that configuration's result (Section 4), not the Hopper row of Table 3.

Environment	N	Unquantized	8-bit	6-bit
Swimmer-v4	2	126.0 ± 1.3	126.1 ± 0.8	128.9 ± 7.4
	4	127.0 ± 0.8	127.1 ± 0.5	125.8 ± 3.4
	8	128.4 ± 0.8	128.6 ± 0.4	127.8 ± 0.7
Humanoid-v4	2	3391 ± 796	2956 ± 593	497 ± 34
	4	5343 ± 121	5244 ± 237	676 ± 73
	8	6329 ± 334	5997 ± 259	1514 ± 526
Hopper-v4 (original config.)	2	2140 ± 386	1901 ± 593	1573 ± 817
	4	1658 ± 273	1755 ± 302	1853 ± 551
	8	1605 ± 159	1713 ± 150	1837 ± 248

Eight bits preserve the result. On Swimmer, 8-bit and unquantized runs are indistinguishable at every agent count. On Humanoid, 8 bits stays within one standard deviation of the unquantized result at $N = 2$ and $N = 4$ and finishes 5% lower at $N = 8$ (5997 ± 259 against 6329 ± 334), also about one standard deviation. Learning still speeds up with the number of agents, which is the paper's claim: at iteration 1000 the smoothed Humanoid return at 8 bits is 580, 905 and 3428 for $N = 2, 4, 8$, against 602, 944 and 3863 unquantized, and the same ordering holds at every checkpoint on both environments. Eight bits therefore removes a further factor of four from the uplink against the `float32` values actually exchanged (eight at the paper's 64-bit accounting), on top of the $\mathcal{O}(d^2) \to \mathcal{O}(d)$ reduction, without giving up either the final reward or the scaling with N.

Six bits are too few. On Humanoid, 6 bits collapses at every agent count, to 497, 676 and 1514. Five seeds put this beyond seed variance and confirm the threshold the single-seed scan found (5543 at 7 bits, 1991 at 6). The ordering in N survives, at about a quarter of the return. Swimmer's final values hold at 6 bits, but the cost shows elsewhere: the seed spread at $N = 2$ and $N = 4$ grows from about 1 to 7.4 and 3.4, and early in training four agents learn *slower* than two (smoothed return 75 against 95 at iteration 100), so the faster-with-more-agents ordering is lost before it is recovered at convergence.

Where the threshold sits. The single-seed scan at $N = 8$ shows a sharp threshold rather than a gradual decay. Swimmer is flat from 6 to 32 bits and collapses to 94.4 at 5; Humanoid holds 5542.6 at 7 bits and falls to 1990.5 at 6. The threshold moves in the direction d does, which is what an unbiased quantizer predicts. The variance injected per element is bounded by $s^2/4$ and enters a d-dimensional sum, so Humanoid's policy ($d \approx 7.3 \times 10^5$) needs one bit more than Swimmer's ($d = 4868$) before the aggregated direction stops degrading. Two environments do not test that reading, and a scan on a fourth would. Taken together with the five-seed grid, 8 bits is the operating point that holds on every environment we tested; 7 bits on Humanoid and 6 on Swimmer are plausible but rest on single seeds or show degraded early learning.

Hopper. The Hopper quantization runs use the practical settings of Swimmer and Humanoid, not the reduced configuration of Section 4, so they are compared with unquantized runs under those same settings (Table 6). At both bit widths and every agent count the quantized runs sit

within the seed spread of the unquantized ones.

7.3 What the byte counts omit

The reported bytes are $d \cdot b/8$ per transmission and exclude the quantization metadata a real deployment would also send: $\min x$ and the scale for the stochastic scheme (two `float32`, 64 bits per transmission). Relative to a $d \cdot b$-bit payload the overhead is $64/(d\,b)$, so roughly 1–3% in the worst case here ($b = 1$, d of a few thousand) and below 0.5% at 8 bits. The downlink broadcast of $\boldsymbol{\theta}$ and $\mathbf{y}$ is not counted either, for the same reason it is not counted in the paper. Neither omission changes the ordering between bit widths, but both make the reported compression slightly optimistic.

Quantizing the baseline instead. A symmetric comparison would quantize standard-average FedNPG's $\mathbf{H}_i$ as well. It does not change the conclusion, and the arithmetic shows why without an experiment. At b bits the standard method uplinks about $d^2 b$ bits per agent per round, against $2 \cdot 64\,d$ for unquantized FedNPG-ADMM, so the two break even only at $b \approx 128/d$, or $64/d$ against `float32`. On Swimmer that is at most 0.03 bits, and smaller still on the larger policies: no bit width makes the $\mathcal{O}(d^2)$ method competitive, and quantizing FedNPG-ADMM widens the gap further.

8 Threats to validity

Our study uses five seeds against the original's ten, so our error bars are wider than they appear relative to a ten-seed estimate. Hopper is reported under a different configuration from Swimmer and Humanoid (Table 8), so comparisons across environments mix the algorithm with its practical settings. All runs use a single machine and a single software stack, so implementation-level differences from the authors' setup cannot be separated from algorithmic ones. That concern has precedent here: the same algorithm under identical hyperparameters has been shown to differ by more than a factor of two on Hopper across implementations [4]. Finally, two hyperparameters (η and the interpretation of Table 4's learning-rate column, Section 4) are our reading of an ambiguous source, and a different reading could change the results, on Hopper in particular.

9 Conclusion

We reproduced FedNPG-ADMM from scratch and evaluated it across three MuJoCo environments, four agent counts and five seeds. Swimmer reproduces the reported final rewards at two, four and eight agents and exceeds them with one, monotone in N, and Humanoid scales 6.4× from one agent to eight. On Hopper, with a reduced configuration closer to the algorithm as written, four and eight agents match the reported rewards and learn faster than one or two, while one and two remain indistinguishable from each other. Against the baselines at eight agents, FedNPG-ADMM matches standard-average FedNPG on Hopper and exceeds it on Humanoid, at three to six orders of magnitude less uplink.

We measured the communication argument directly instead of assuming it, obtaining per-agent per-round reductions of 2434×, 9219× and 363537× on the three environments. FedNPG-ADMM is not the cheapest of the compared methods in absolute terms, since it transmits twice what FedPPO does, so its case rests on convergence per round.

Quantizing the agent-to-server transmissions, which the original work does not attempt, costs nothing measurable at 8 bits. Across five seeds, Swimmer and Humanoid keep their final reward and their faster learning with more agents, for a further 4× against the `float32` values exchanged (8× at the paper's 64-bit accounting) on top of the $\mathcal{O}(d^2) \to \mathcal{O}(d)$ reduction. Below

a threshold that grows with the policy dimension the collapse is abrupt rather than graceful: Humanoid fails at 6 bits at every agent count, while Swimmer holds its final values there but loses the agent-count ordering early in training.

Limitations and future work. The missing axis is communication *frequency*. FedNPG-ADMM optimizes what is transmitted per round, while FedAvg-style schemes with E local steps optimize how often rounds occur [5, 7]. The two are orthogonal and could be combined. A second open question is the Humanoid gap between FedNPG-ADMM and the exact aggregation it approximates (Section 6).

References

[1] Joshua Achiam. Spinning up in deep reinforcement learning: Benchmarks. `https://spinningup.openai.com/en/latest/spinningup/bench.html`, 2018.

[2] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[3] Anis Elgabli, Jihong Park, Amrit S. Bedi, Chaouki Ben Issaid, Mehdi Bennis, and Vaneet Aggarwal. Q-GADMM: Quantized group ADMM for communication efficient decentralized machine learning. *IEEE Transactions on Communications*, 69(1):164–181, 2021.

[4] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

[5] Hao Jin, Yang Peng, Wenhao Yang, Shusen Wang, and Zhihua Zhang. Federated reinforcement learning with environment heterogeneity. In *Artificial Intelligence and Statistics (AISTATS)*, 2022.

[6] Sham Kakade. A natural policy gradient. In *Advances in Neural Information Processing Systems (NIPS)*, 2001.

[7] Sajad Khodadadian, Pranay Sharma, Gauri Joshi, and Siva Theja Maguluri. Federated reinforcement learning: Linear speedup under markovian sampling. In *International Conference on Machine Learning (ICML)*, 2022.

[8] Guangchen Lan, Han Wang, James Anderson, Christopher Brinton, and Vaneet Aggarwal. Improved communication efficiency in federated natural policy gradient via ADMM-based gradient updates. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2310.19807.

[9] Aritra Mitra, George J. Pappas, and Hamed Hassani. Temporal difference learning with compressed updates: Error-feedback meets reinforcement learning. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2301.00944.

[10] Jiaju Qi, Qihao Zhou, Lei Lei, and Kan Zheng. Federated reinforcement learning: Techniques, applications, and open challenges. *Intelligence & Robotics*, 1(1):18–57, 2021. arXiv:2108.11887.

[11] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, 2015.

[12] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[13] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 2000.

[14] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.

A Hyperparameters

Table 7: Values transcribed from Table 4 of Lan et al. [8]. $\lambda_{\mathrm{GAE}} = 0.95$ comes from the paper's setup text rather than the table.

	Swimmer-v4	Hopper-v4	Humanoid-v4
ADMM penalty ρ	0.1	0.1	0.01
Trust region δ	0.01	0.01	0.01
Discount γ	0.99	0.99	0.99
Timesteps per iteration T	2048	1024	512
Iterations K	1000	2000	3000
Learning rate (critic)	3×10^{-4}	3×10^{-4}	10^{-5}
Hidden layers	64×64	128×128	512^3
Hidden activation	ReLU	ReLU	ReLU
Policy mean output	Tanh	Tanh	Tanh

Table 8: Settings not specified by the paper, per environment. Swimmer and Humanoid follow the practical basis the paper's setup section cites (TRPO with line search, and stable-baselines3). Hopper uses a reduced set, closer to Algorithm 1 as written (Section 4).

Setting	Swimmer, Humanoid	Hopper	Basis
Obs. normalization	running mean/std, clip 10	off	SB3 `VecNormalize`
Reward normalization	scale by return std, clip 10	off	SB3 `VecNormalize`
Adv. normalization	per agent, $(a - \mu)/(\sigma + 10^{-8})$	off	SB3
Line search	10 backtracks, shrink 0.8	off	TRPO
$\log \sigma$ clamp	$[-2.0, 0.5]$	off	ours
Orthogonal init.	gain $\sqrt{2}$ hidden, 0.01 policy, 1.0 value		SB3
$\log \sigma$ initial	-0.5		tianshou, Spinning Up
Critic fitting	10 epochs, minibatch 64, Adam		SB3 PPO defaults
CG iters / damping	10 / ρ		TRPO, line 8
η	1.0		not specified
Seeds	0–4		—